

Parte 2

Hay 2 versiones de cada algoritmo:

Una versión sin instrumentación, para medir el tiempo real con `time.perf_counter`. Para “n” pequeños, la llamada se repite en lotes hasta acumular al menos 0.2 s y se reporta la mediana de 5 lotes.

Una versión instrumentada, que incrementa un contador en cada operación según el modelo de costo de la Parte 1.

Validación: en cada corrida, el conteo instrumentado se compara con la fórmula $T(n)$ de la Parte 1 mediante un `assert`. Coinciden exactamente en todos los casos ejecutados.

Salida: el `printf` se reemplaza por una escritura a `os.devnull`. Así, la operación de salida se ejecuta, pero no se mide la velocidad de la terminal.

Corridas inviables: tres combinaciones requieren entre 10^{11} y 10^{13} operaciones y tardarían horas en Python (marcadas con *):

- o (a, $n = 10^5$)

- o (a, $n = 10^6$)

- o (c, $n = 10^6$)

Para ellas, el conteo se obtiene con la fórmula (ya validada). El tiempo se estima como $\text{operaciones} \div (\text{operaciones por segundo medidas en la corrida real más grande de ese algoritmo})$.

Resultados

n	Algoritmo a — $O(n^2 \log n)$		Algoritmo b — $O(n)$		Algoritmo c — $O(n^2)$	
	Operaciones	Tiempo	Operaciones	Tiempo	Operaciones	Tiempo
1	25	0.27 μ s	1	0.04 μ s	2	0.05 μ s
10	507	4.38 μ s	63	2.39 μ s	41	2.37 μ s
100	63 957	524.22 μ s	603	22.44 μ s	2 609	190.10 μ s
1 000	8 519 007	79.36 ms	6 003	220.02 μ s	251 084	18.93 ms
10 000	1 150 250 007	10.25 s	60 003	2.30 ms	25 010 834	1.87 s
100 000	137 502 950 007	1224.71 s*	600 003	22.36 ms	2 500 108 334	183.59 s
1 000 000	16 000 034 000 007	39.6 h*	6 000 003	226.54 ms	250 001 083 334	5.1 h*

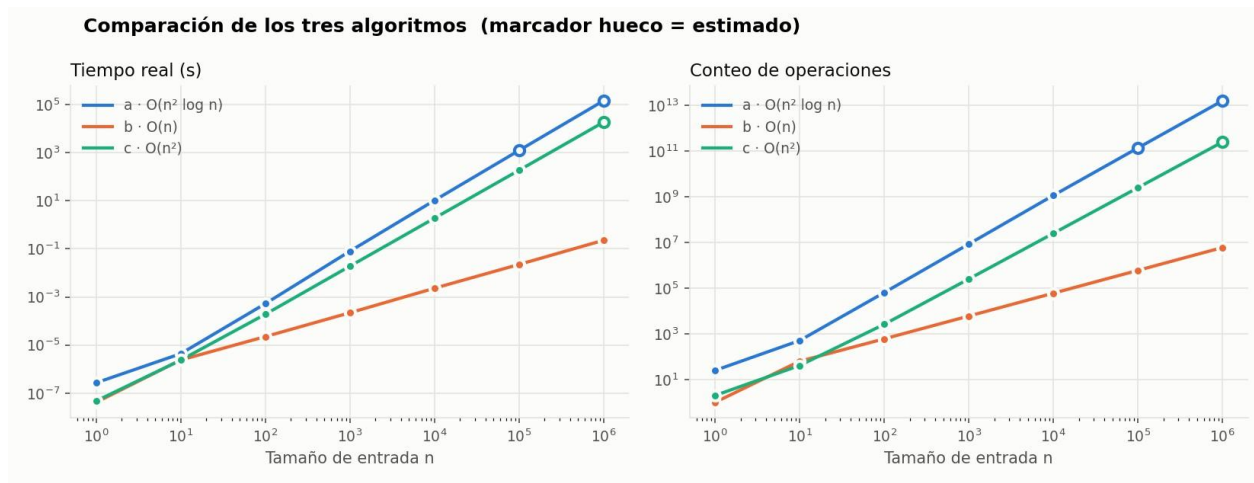


Figura 1. Tiempo real y conteo de operaciones de los tres algoritmos (escala log-log). Marcador hueco = estimado.

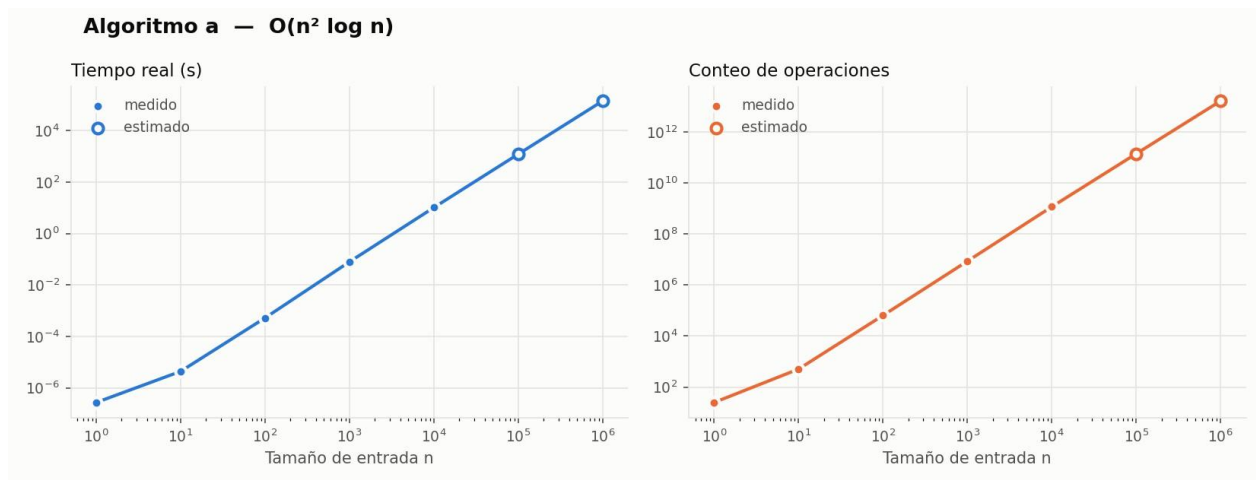


Figura 2. Algoritmo a: tiempo real frente a conteo de operaciones.

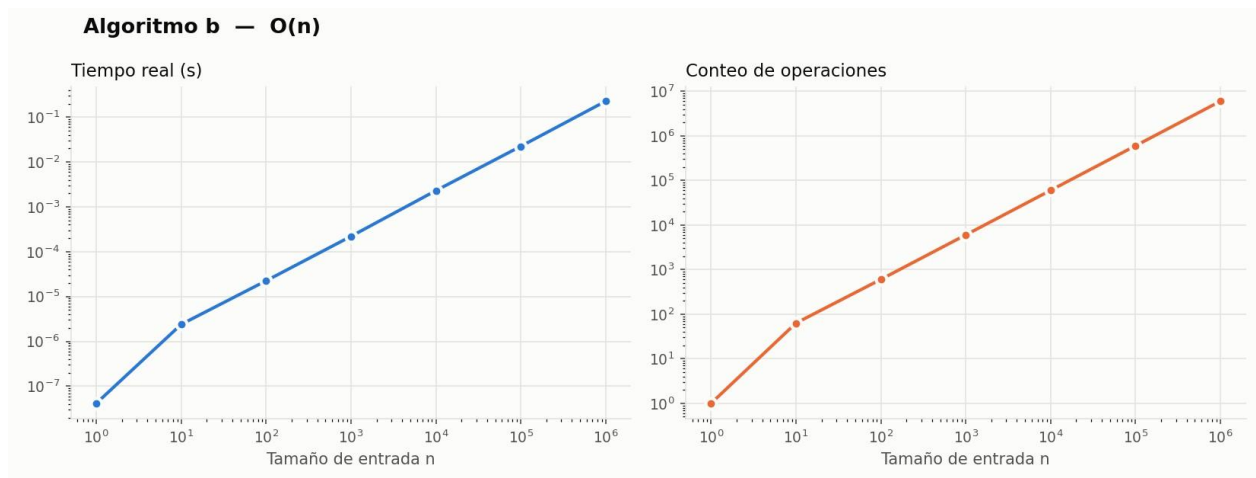


Figura 3. Algoritmo b: tiempo real frente a conteo de operaciones.

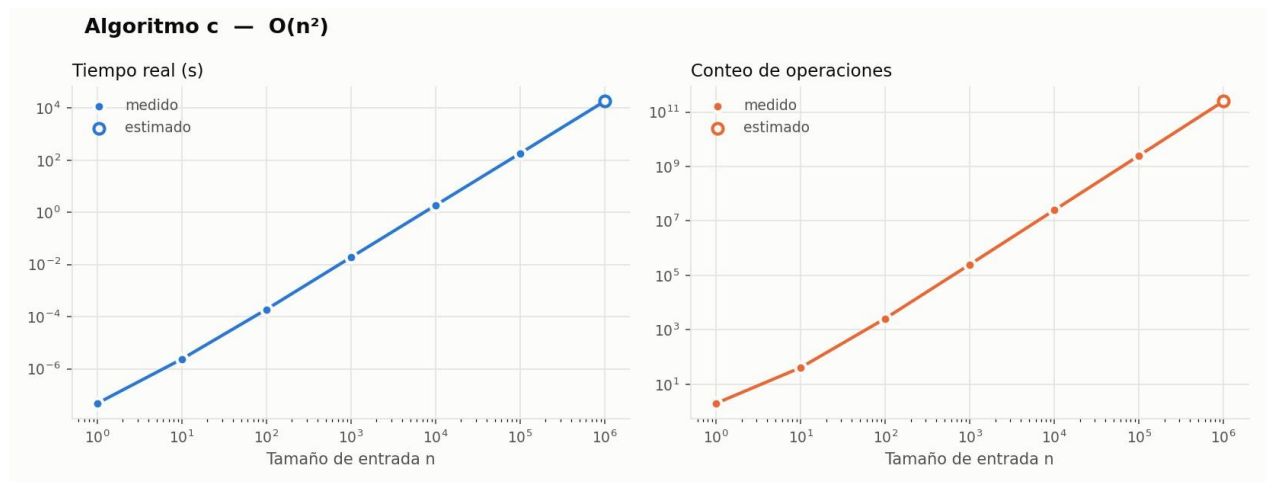


Figura 4. Algoritmo c: tiempo real frente a conteo de operaciones.